

WHY MATH & GEOMETRY?

Matrix transformations (rotations, spirals, zeroing) and numerical algorithms (binary exponentiation, digit manipulation) rely on **pure mathematical symmetries** instead of complex data structures!

THE 90° ROTATION AHA

Rotating an $N \times N$ matrix 90° clockwise in-place is mathematically equivalent to:

- Transpose Matrix** ('swap(matrix[i][j], matrix[j][i])')
- Reverse Each Row** ('swap(matrix[i][l], matrix[i][r])')

3 CORE GEOMETRIC PATTERNS

- Matrix Transpose & Reverse:** 90° clockwise rotation in $O(N^2)$ time and $O(1)$ space.
- 4-Boundary Spiral Scan:** Shrink 'top', 'bottom', 'left', 'right' boundaries after scanning each edge.
- First Row/Col State Markers:** Mark zeroes in row 0 and col 0 for $O(1)$ auxiliary space in Set Matrix Zeroes!

VISUALIZING MATRIX ROTATION

Original → Transpose → Reversed Rows:

Original:	Transpose:	Reversed Rows (90°):
$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$	$\begin{bmatrix} 3 & 1 \\ 4 & 2 \end{bmatrix}$

PREREQUISITES & COMPLEXITY REASONING

Algorithm / Operation	Time Complexity	Auxiliary Space
Rotate Image (Transpose + Reverse)	$O(N^2)$	$O(1)$ in-place
Spiral Matrix (4 Boundaries)	$O(M \cdot N)$	$O(1)$ space
Set Matrix Zeroes (Row/Col 0 Markers)	$O(M \cdot N)$	$O(1)$ space
Pow(x, n) (Binary Exponentiation)	$O(\log N)$	$O(\log N)$ stack

CLUES THAT SCREAM MATH & GEOMETRY

- "Rotate 2D matrix 90 degrees in-place" → Transpose + Reverse Rows
- "Traverse 2D matrix in spiral order" → 4 Boundary Pointers ('top', 'bot', 'left', 'right')
- "Compute x^n in $O(\log n)$ time" → Binary Exponentiation ('n / 2' recursive squarings)

MATRIX TRANSPOSE HELPER

```
public static void transpose(int[][] matrix) {
    int n = matrix.length;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
}
```

BINARY EXPONENTIATION ('Pow(x, n)')

```
public double myPow(double x, int n) {
    long N = n;
    if (N < 0) { x = 1 / x; N = -N; }
    return fastPow(x, N);
}

private double fastPow(double x, long n) {
    if (n == 0) return 1.0;
    double half = fastPow(x, n / 2);
    if (n % 2 == 0) return half * half;
    else return half * half * x;
}
```

>

PATTERN 1

ROTATE IMAGE (Transpose + Row Reversal)

e.g. Rotate Image (LC 48)

WHEN TO USE

Rotate $N \times N$ 2D matrix 90° clockwise in-place ($O(1)$ space).

TEMPLATE — LC 48

```
public void rotate(int[][] matrix) {
    int n = matrix.length;
    // Step 1: Transpose matrix (swap matrix[i][j] with matrix[j][i])
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int tmp = matrix[i][j]; matrix[i][j] = matrix[j][i]; matrix[j][i] = tmp;
        }
    }
    // Step 2: Reverse each row
    for (int i = 0; i < n; i++) {
        for (int l = 0, r = n - 1; l < r; l++, r--) {
            int tmp = matrix[i][l]; matrix[i][l] = matrix[i][r]; matrix[i][r] = tmp;
        }
    }
}
```

PATTERN 2

SPIRAL MATRIX (4-Boundary Shrinking Loop)

e.g. Spiral Matrix (LC 54)

BOUNDARY SHRINK LOGIC

Traverse top row, right col, bottom row, left col.
Increment 'top++', 'left++', decrement 'bot--', 'right--'!

TEMPLATE — LC 54

```
public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> res = new ArrayList<>();
    int top = 0, bot = matrix.length - 1, left = 0, right = matrix[0].length - 1;
    while (top <= bot && left <= right) {
        for (int c = left; c <= right; c++) res.add(matrix[top][c]); top++;
        for (int r = top; r <= bot; r++) res.add(matrix[r][right]); right--;
        if (top <= bot) { for (int c = right; c >= left; c--) res.add(matrix[bot][c]); bot--; }
        if (left <= right) { for (int r = bot; r >= top; r--) res.add(matrix[r][left]); left++; }
    }
    return res;
}
```

>

PATTERN 3

SET MATRIX ZEROES (First Row/Col Marker $O(1)$ Space)

e.g. Set Matrix Zeroes (LC 73)

IN-PLACE MARKER TRICK

Use 'matrix[r][0]' and 'matrix[0][c]' as flag markers!
Track 'rowZero' and 'colZero' booleans for the first row and column.

TEMPLATE — LC 73

```
public void setZeroes(int[][] matrix) {
    int rows = matrix.length, cols = matrix[0].length;
    boolean rowZero = false, colZero = false;
    for (int r = 0; r < rows; r++) if (matrix[r][0] == 0) colZero = true;
    for (int c = 0; c < cols; c++) if (matrix[0][c] == 0) rowZero = true;
    for (int r = 1; r < rows; r++)
        for (int c = 1; c < cols; c++)
            if (matrix[r][c] == 0) { matrix[r][0] = 0; matrix[0][c] = 0; }
    for (int r = 1; r < rows; r++)
        for (int c = 1; c < cols; c++)
            if (matrix[r][0] == 0 || matrix[0][c] == 0) matrix[r][c] = 0;
    if (colZero) for (int r = 0; r < rows; r++) matrix[r][0] = 0;
    if (rowZero) for (int c = 0; c < cols; c++) matrix[0][c] = 0;
}
```

>

PATTERN 5

PLUS ONE (Backwards Carry Propagation)

e.g. Plus One (LC 66)

CARRY PROPAGATION

Scan backwards. If digit ≥ 9 , increment and return! If all digits were 9 (e.g. 999), return array `[1, 0, 0, 0]`.

TEMPLATE — LC 66

```
public int[] plusOne(int[] digits) {
    for (int i = digits.length - 1; i >= 0; i--) {
        if (digits[i] < 9) { digits[i]++; return digits; }
        digits[i] = 0;
    }
    int[] res = new int[digits.length + 1];
    res[0] = 1;
    return res;
}
```

PATTERN 6

MULTIPLY STRINGS (Position Index Product Array)

e.g. Multiply Strings (LC 43)

POSITION MATH ('I + J + 1')

Product of `num1[i]` and `num2[j]` maps to positions `p1 = i + j` and `p2 = i + j + 1` in result array of size `M + N`.

TEMPLATE — LC 43

```
public String multiply(String num1, String num2) {
    int m = num1.length(), n = num2.length();
    int[] pos = new int[m + n];
    for (int i = m - 1; i >= 0; i--) {
        for (int j = n - 1; j >= 0; j--) {
            int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0');
            int p1 = i + j, p2 = i + j + 1;
            int sum = mul + pos[p2];
            pos[p1] += sum / 10;
            pos[p2] = sum % 10;
        }
    }
    StringBuilder sb = new StringBuilder();
    for (int p : pos) if (!(sb.length() == 0 && p == 0)) sb.append(p);
    return sb.length() == 0 ? "0" : sb.toString();
}
```

>

PATTERN 7

DETECT SQUARES (Diagonal Point Matching)

e.g. Detect Squares (LC 2013)

DIAGONAL ALIGNMENT MATCH

For query point `(px, py)`, iterate over recorded points `(x, y)`. If `|x - px| == |y - py| > 0` (diagonal!), count matches of `(px, y)` and `(x, py)`!

TEMPLATE — LC 2013

```
class DetectSquares {
    private List<int[]> points = new ArrayList<>();
    private Map<String, Integer> count = new HashMap<>();

    public void add(int[] point) {
        points.add(point);
        String key = point[0] + "@" + point[1];
        count.put(key, count.getDefault(key, 0) + 1);
    }

    public int count(int[] point) {
        int px = point[0], py = point[1], ans = 0;
        for (int[] pt : points) {
            int x = pt[0], y = pt[1];
            if (Math.abs(px - x) != Math.abs(py - y) || px == x || py == y) continue;
            ans += count.getDefault(px + "@" + y, 0) * count.getDefault(x + "@" + py, 0);
        }
        return ans;
    }
}
```

>

MATH & GEOMETRY — WHICH PATTERN TO USE?

Start: What is the input type & operation?

Rotate 2D matrix 90° clockwise

→

Transpose then + Reverse reverse rows

Spiral traversal of matrix

→

4 Boundary Pointers Shrink 'top', 'bot', 'left', 'right'

Compute x^n in O(log n) time

→

Binary Exponentiation fast n / 2

Count axis-aligned squares

→

Diagonal Point Match 'x - px == |y - py| > 0'

Trigger Word	Variant	Action
"Rotate matrix 90 deg"	Transpose + Reverse	<code>`swap(i, j)`</code> , reverse rows
"Spiral matrix"	4 Boundaries	Loop 4 edges, shrink bounds
"Set matrix zeroes O(1) space"	First Row/Col Markers	Use row 0 & col 0 as flags
"Pow(x, n) fast"	Binary Exponentiation	Recurse <code>`n / 2`</code> squarings

Interviewer Asks	Your Answer
How to rotate 90° COUNTER-clockwise?	Either Transpose then Reverse Columns, OR Reverse Rows first then Transpose!
How to handle <code>`n = -2147483648`</code> in <code>`Pow(x, n)`</code> ?	Cast <code>`n`</code> to <code>`long N = n`</code> to prevent integer overflow when taking <code>`-n`</code> !

EASY & MEDIUM — Array Math & Rotations

LC 66 · Plus OneEASY

Pattern: Backwards Carry
Key Insight: Increment and return if digit ≤ 9 .

```
public int[] plusOne(int[] digits) {
    for (int i = digits.length - 1; i >= 0; i--) {
        if (digits[i] < 9) { digits[i]++; return digits; }
        digits[i] = 0;
    }
    int[] res = new int[digits.length + 1]; res[0] = 1;
    return res;
}
```

LC 48 · Rotate ImageMEDIUM

Pattern: Transpose + Reverse
Key Insight: 'swap(i,j)' then reverse each row.

```
public void rotate(int[][] m) {
    int n = m.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
        }
    for (int i = 0; i < n; i++)
        for (int l = 0, r = n - 1; l < r; l++, r--) {
            int t = m[i][l]; m[i][l] = m[i][r]; m[i][r] = t;
        }
}
```

LC 54 · Spiral MatrixMEDIUM

Pattern: 4-Boundary Loop
Key Insight: Shrink 'top', 'bot', 'left', 'right'.

```
public List<Integer> spiralOrder(int[][] m) {
    List<Integer> res = new ArrayList<>();
    int t = 0, b = m.length - 1, l = 0, r = m[0].length - 1;
    while (t <= b && l <= r) {
        for (int c = l; c <= r; c++) res.add(m[t][c]); t++;
        for (int row = t; row <= b; row++) res.add(m[row][r]); r--;
        if (t <= b) { for (int c = r; c >= l; c--) res.add(m[b][c]); b--; }
        if (l <= r) { for (int row = b; row >= t; row--) res.add(m[row][l]); l++; }
    }
    return res;
}
```

MEDIUM — String Multiplication

LC 43 · Multiply StringsMEDIUM

Pattern: Product Position Map
Key Insight: 'num1[i] * num2[j]' maps to 'i+j' and

```
public String multiply(String num1, String num2) {
    int m = num1.length(), n = num2.length();
```

MEDIUM — Matrix Zeroes & Pow(x,n)

LC 73 · Set Matrix ZeroesMEDIUM

Pattern: First Row/Col Marker
Key Insight: Use row 0 & col 0 as flag storage.

```
public void setZeroes(int[][] m) {
    int R = m.length, C = m[0].length;
    boolean r0 = false, c0 = false;
    for (int r = 0; r < R; r++) if (m[r][0] == 0) c0 = true;
    for (int c = 0; c < C; c++) if (m[0][c] == 0) r0 = true;
    for (int r = 1; r < R; r++)
        for (int c = 1; c < C; c++)
            if (m[r][c] == 0) { m[r][0] = 0; m[0][c] = 0; }
    for (int r = 1; r < R; r++)
        for (int c = 1; c < C; c++)
            if (m[r][0] == 0 || m[0][c] == 0) m[r][c] = 0;
    if (c0) for (int r = 0; r < R; r++) m[r][0] = 0;
    if (r0) for (int c = 0; c < C; c++) m[0][c] = 0;
}
```

LC 50 · Pow(x, n)MEDIUM

Pattern: Binary Exponentiation
Key Insight: 'fastPow(x, n/2)'. Cast 'long N = n'.

```
public double myPow(double x, int n) {
    long N = n; if (N < 0) { x = 1 / x; N = -N; }
    return fastPow(x, N);
}
private double fastPow(double x, long n) {
    if (n == 0) return 1.0;
    double half = fastPow(x, n / 2);
    return (n % 2 == 0) ? half * half : half * half * x;
}
```

MEDIUM — Detect Squares Geometry

LC 213 · Detect SquaresMEDIUM

Pattern: Diagonal Match
Key Insight: Match '|px - x| == |py - y| > 0'.

```
public int count(int[] point) {
    int px = point[0], py = point[1], ans = 0;
    for (int[] pt : points) {
```

DRY RUN — Rotate Image 90° Clockwise

Step	Input State	Operation	Output State
1	<code>[[1,2], [3,4]]</code>	Transpose <code>swap(matrix[0][1], matrix[1][0])</code>	<code>[[1,3], [2,4]]</code>
2	<code>[[1,3], [2,4]]</code>	Reverse row 0: <code>[1, 3] → [3, 1]</code>	<code>[[3,1], [2,4]]</code>
3	<code>[[3,1], [2,4]]</code>	Reverse row 1: <code>[2, 4] → [4, 2]</code>	<code>[[3,1], [4,2]]</code>

MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Rotate matrix 90° in-place with Transpose + Reverse
- ☐ Code Spiral Matrix 4-boundary loop
- ☐ Set Matrix Zeroes in O(1) space with row/col 0 flags
- ☐ Code Pow(x, n) via binary exponentiation
- ☐ Write Happy Number with Floyd's fast/slow digits
- ☐ Code Plus One backwards carry propagation
- ☐ Write Multiply Strings position index mapping
- ☐ Prove why Transpose + Reverse = 90° Rotation

MATHEMATICAL PROOF — 90° CLOCKWISE MATRIX ROTATION

Claim: Transposing a matrix and reversing each row rotates the matrix 90° clockwise.

Proof: Transpose maps element $(r, c) \rightarrow (c, r)$. Reversing rows maps element $(c, r) \rightarrow (c, N - 1 - r)$. Under 90° clockwise rotation, original coordinate (r, c) moves to $(c, N - 1 - r)$ **Exact Mathematical Identity!**

GOLDEN RULES — NEVER FORGET

- Rule 1 — Cast Negative Powers to Long**
- In `Pow(x, n)`, if `n = -2147483648`, taking `-n` causes 32-bit signed integer overflow! Cast to `long N = n` before negation!
- Rule 2 — Guard Extra Loops in Spiral Matrix**
- In Spiral Matrix, check `if (top <= bot)` before scanning bottom row and `if (left <= right)` before scanning left column to prevent duplicate row scans on odd dimensions!